



AI 기반 심전도 부정맥 판별

한림대학교 SW Capstone 결과보고

팀: 이공이공 | 지도: 안재목 교수

참여: 방정우, 권기웅, 윤주원

목차

01 과제 배경 및 목적

- 고령화 사회와 심혈관 질환 증가
- 부정맥 조기 진단의 중요성
- 기존 ECG 판독 방식의 한계
- AI 진단 보조 시스템 필요성

02 문제점 및 해결 방안

- 상시 모니터링 부족 문제
- 모델 연구와 실무 괴리
- CNN과 Streamlit 연동 해결
- 통합 파이프라인 구축

03 시스템 구조 및 데이터 전처리

- MIT-BIH 데이터셋 활용
- 10초 단위 window 분할
- Conv1D 기반 CNN 설계
- 5가지 모델 비교 실험

04 CNN 모델 설계 및 학습

- 시계열 특징 추출 모델
- 정규화 및 클래스 균형
- 다양한 평가 지표 적용
- 최적 모델 선정 및 저장

05 웹 시스템 구현 및 특징점

- Streamlit 기반 웹 UI
- 실시간 ECG 시각화
- 파일 업로드 분석 기능
- 부정맥 위험도 제공

06 활용 방안 및 기대효과

- 디지털 헬스케어 확장
- 의료진 판독 부담 경감
- 즉각 배포 가능한 실무 가치
- 상용화 가능성 제시

과제 개요 및 배경

본 과제는 AI 기반 심전도 분석을 통한 부정맥 자동 판별 시스템 개발을 목표로 진행되었습니다.

팀 구성

- 팀명: 이공이공
- 지도교수: 안재목
- 참여 학생:
방정우(20205182, 대표)
권기웅(20205114, 빅데이터)
윤주원(20203231, 디지털금융정보)

참여 분야 및 기업

- 참여 분야: AI
- 참여 기업: ㈜안랩
- Github 저장소를 통한 오픈소스 공개

핵심 기술 스택

- TensorFlow/Keras: CNN 모델 구현
- Streamlit: 웹 UI 구현
- NumPy/Pandas: 데이터 처리
- Plotly/Matplotlib: 시각화
- WFDB: ECG 데이터 로드

데이터셋

- MIT-BIH Arrhythmia Database
- PhysioNet 플랫폼 제공
- 환자 5명의 ECG 신호
- 부정맥 학습용 공개 데이터셋
- Annotation 처리 지원

연구 목적 및 필요성

부정맥은 조기 발견 시 치료 가능성이 높으나, 기존 수동 판독은 시간과 비용 부담이 큼.

기존 방식의 한계

- 대용량 데이터 분석 부담
- 판독자 숙련도 의존
- 진단 일관성 저해

고령화로 심혈관 질환과 치명적 합병증 유발
부정맥의 조기 진단·상시 모니터링이 중요합니다.

AI 시스템 도입 필요성

- 복잡한 패턴 정밀 추출
- 대량 의료 데이터 처리
- 딥러닝 기반 자동화

본 과제의 목표

- MIT-BIH 기반 분류
- 1D-CNN 모델 설계
- Streamlit 웹 연동

기존 기술의 문제점

기존 ECG 분석 기술은 병원 중심으로만 운영되어 실시간 확인이 어렵고, 일반 사용자의 접근성이 매우 낮습니다.



상시 모니터링 및 직관성 부족

- 병원·의료기관 중심 분석
- 실시간 확인 불가능
- 전문적 파형 구조 특성
- 일반 사용자 이해 어려움
- 직관적 인터페이스 부재

모델 연구와 실무 시스템 간 괴리

- 알고리즘 학습에만 편중
- 수치적 성능 평가 중심
- 실제 사용자 접근 불가
- 서비스 형태 시스템 부재
- 연구와 실무 간 단절

본 과제의 해결 방안

의료 데이터 구축부터 AI 모델링, 웹 서비스 배포까지 전 과정을 통합하였습니다.

통합 파이프라인 구축

- MIT-BIH 데이터 파싱
- 10초 window 분할
- 균형 잡힌 학습셋 구성

학습된 CNN과 Streamlit 연동으로
파형 시각화와 부정맥 위험도 수치화를 제공하
는 사용자 중심 ECG 플랫폼을 구축했습니다.

AI 모델과 웹 연동

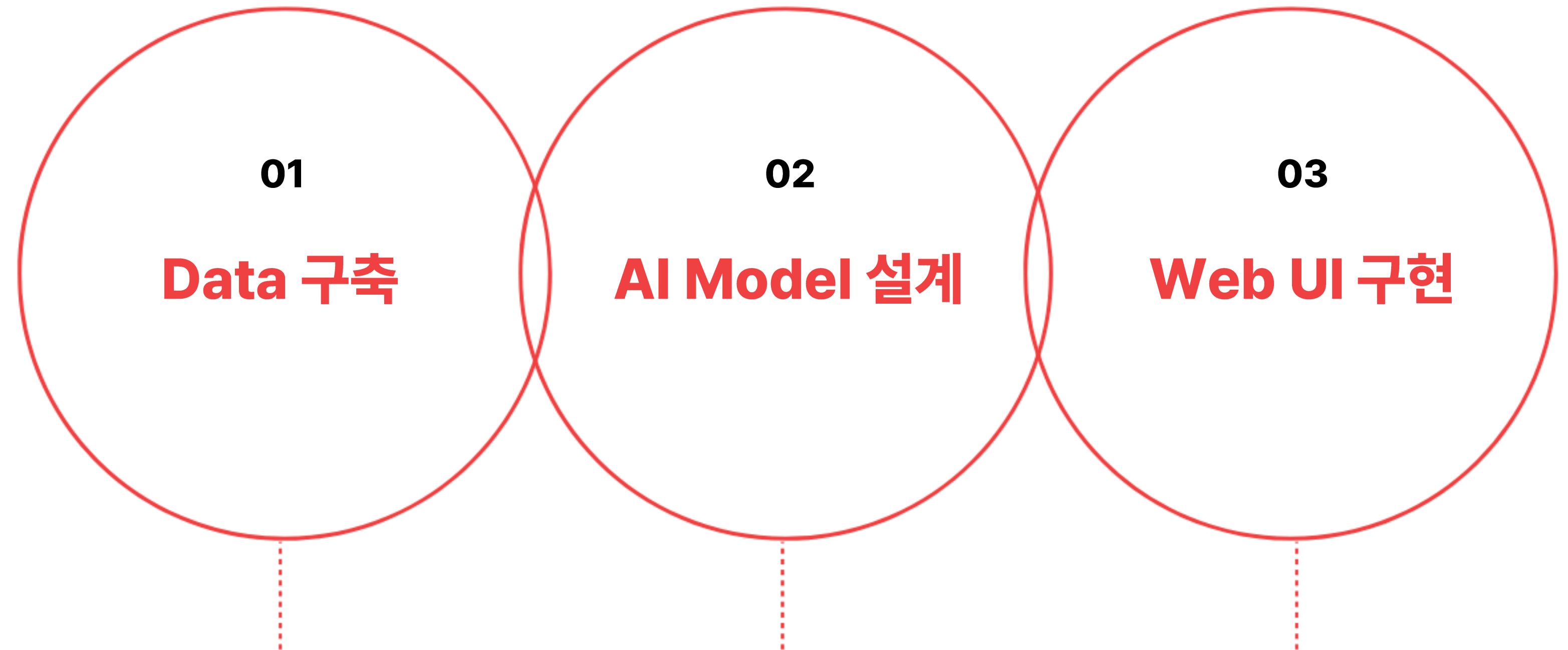
- Conv1D 기반 CNN
- 5가지 구조 비교
- Streamlit UI 구현

사용자 중심 설계

- 파형 시각화
- 위험도 수치화 제공
- 외부 파일 업로드 지원

시스템 파이프라인 구조

MIT-BIH 공개 데이터셋을 파싱하여 10초 단위 window로 분할함으로써 균형 잡힌 학습 데이터를 구성하였습니다.



- Conv1D 기반 5가지 CNN 구조 설계
- 최적 분류 모델 선정
- 시계열 파형 특징 추출 특화

- Streamlit 기반 Plotly 동적 시각화
- 외부 파일 업로드 분석 기능
- 실시간 ECG 그래프 출력

- 의료 데이터 구축부터 배포까지 통합
- 사용자 중심 분석 플랫폼
- 즉각 배포 가능한 시스템

본 시스템은 데이터 수집부터 AI 모델링, 웹 서비스 배포까지 통합된 end-to-end 파이프라인으로 구성되어 있습니다.

데이터 수집 및 전처리 (1)

환자 5명(200, 208, 210, 214, 221
번)의 ECG 데이터를 MIT-BIH
Arrhythmia Database에서 추출하
여 분석에 활용하였습니다.

MIT-BIH Database 활용

- 1채널 ECG 사용
- 연속 신호 추출
- 표준화된 데이터 형식

**MIT-BIH Arrhythmia Database를 활용
하여 환자 5명의 ECG 데이터를 수집하고
10초 단위로 분할하였습니다.**

ECG 신호 추출

- 10초 단위 window 분할 적용
- 환자당 50개 window 생성
- 총 250개 ECG 샘플 구성

Window 분할 방식

- CNN 모델 입력 데이터로 활용
- 일정한 길이의 구간 분할
- 학습 효율성 극대화

데이터 수집 및 전처리 (2)

PVC 개수 0~2개는 정상(label=0),
3개 이상은 부정맥(label=1)으로 분
류하여 명확한 라벨링 기준을 수립하
였습니다.

라벨링 기준 설정

- 정상 데이터: 128개 (51.2%)
- 부정맥 데이터: 122개 (48.8%)
- 비교적 균형 잡힌 클래스 분포

**PVC 개수를 기준으로 정상과 부정맥을 라
벨링하여 균형 잡힌 데이터셋을 구성하였습
니다.**

클래스 분포 분석

- 모델 편향 문제 최소화
- 특정 클래스 과적합 방지
- 안정적인 학습 환경 조성

데이터 저장 형식

- X_ecg.npy: ECG 신호 데이터
 - y_ecg.npy: 라벨 데이터
- ecg_dataset.csv: 상세 정보

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	record	window_n	start_samp	end_samp	pvc_count	label	0	1	2	3	4	5	6	7
2	200	1	0	3600	8	1	0.35	0.35	0.35	0.35	0.35	0.35	0.35	0.35
3	200	2	10800	14400	4	1	-0.18	-0.19	-0.19	-0.185	-0.19	-0.19	-0.19	-0.2
4	200	3	25200	28800	8	1	0.035	0.035	0.045	0.055	0.05	0.025	0	-0.02
5	200	4	36000	39600	8	1	-0.1	-0.095	-0.115	-0.11	-0.12	-0.105	-0.105	-0.105
6	200	5	50400	54000	0	0	-0.165	-0.17	-0.185	-0.22	-0.23	-0.25	-0.26	-0.27
7	200	6	64800	68400	7	1	-0.08	-0.065	-0.03	-0.02	-0.035	-0.055	-0.08	-0.065
8	200	7	75600	79200	4	1	-0.055	-0.05	-0.045	-0.035	-0.03	-0.03	-0.05	-0.02
9	200	8	90000	93600	2	0	0.17	0.24	0.305	0.4	0.505	0.61	0.725	0.86
10	200	9	104400	108000	3	1	-0.215	-0.215	-0.22	-0.215	-0.2	-0.175	-0.175	-0.195
11	200	10	115200	118800	6	1	-0.39	-0.4	-0.35	-0.23	-0.08	0.06	0.175	0.32
12	200	11	129600	133200	3	1	-0.08	-0.09	-0.085	-0.05	-0.065	-0.145	-0.22	-0.235
13	200	12	144000	147600	4	1	-0.55	-0.54	-0.54	-0.53	-0.485	-0.43	-0.36	-0.32
14	200	13	154800	158400	7	1	1.095	1.14	1.075	0.905	0.57	0.12	-0.28	-0.505
15	200	14	169200	172800	3	1	-2.135	-2.02	-1.92	-1.855	-1.775	-1.695	-1.58	-1.45
16	200	15	183600	187200	4	1	-0.2	-0.2	-0.185	-0.155	-0.15	-0.155	-0.175	-0.17
17	200	16	194400	198000	0	0	-0.205	-0.22	-0.225	-0.235	-0.21	-0.23	-0.25	-0.26
18	200	17	208800	212400	7	1	-0.33	-0.335	-0.33	-0.305	-0.29	-0.305	-0.335	-0.32
19	200	18	223200	226800	1	0	-0.03	-0.02	-0.03	-0.025	-0.025	-0.01	0.01	0.025
20	200	19	234000	237600	6	1	-0.03	-0.025	-0.015	-0.025	-0.02	-0.035	-0.07	-0.085
21	200	20	248400	252000	4	1	-0.325	-0.345	-0.34	-0.31	-0.275	-0.275	-0.29	-0.3
22	200	21	262800	266400	2	0	-0.21	-0.21	-0.195	-0.185	-0.195	-0.195	-0.2	-0.205
23	200	22	273600	277200	0	0	-0.225	-0.21	-0.205	-0.22	-0.235	-0.24	-0.23	-0.215

ecg_dataset.csv



데이터 확인 및 분석을 위해 다음과 같은 칼럼들을 포함한 **ecg_dataset.csv** 파일도 함께 생성.

- record : 환자 번호
- window_number : 윈도우 번호(개수)
- start_sample : 시작 샘플
- end_sample : 끝 샘플
- pvc_count : PVC 개수
- label : 0(정상), 1(부정맥)
- 0,1,2,... : 모델의 입력 피쳐(Feature)로 사용되는 연속적인 심전도(ECG) 파형 진폭 값



pvc_count



record



label



window_number

CNN 기반 부정맥 분류 모델

ECG는 시간에 따른 전압 변화의 1차원 시계열 데이터로, 연속적인 흐름과 반복적인 파형 패턴을 동시에 가지고 있습니다.

ECG 신호 특성

- 1차원 시계열 데이터 구조
- 시간축 기반 전압 변화
- QRS complex 등 주요 파형

ECG 신호의 시계열 특성을 고려하여
Conv1D 기반 CNN 모델을 설계하였습니다.

Conv1D 적용

- 시간축을 따라 이동하며 특징 추출
 - 지역적 파형 패턴 인식
 - 부정맥 분석에 최적화

패턴 학습 전략

- QRS complex 심장 박동 패턴 학습
 - 부정맥 발생 시 파형 변화 탐지
 - 형태 및 간격 변화 분석

데이터 정규화 및 전처리



**모델 학습 전 ECG 신호를 정규화하고
입력 형태를 변환.**

- 정규화 방식: $X = (X - X.mean()) / (X.std() + 1e-8)$
- 환자마다 다른 진폭 범위를 평균과 표준편차로 표준화
- Conv1D 입력 형태로 변환: (샘플 수, 신호 길이, 채널 수)
- Train/Test 분리: 80%/20%, stratify 옵션으로 클래스 비율 유지
- Class weight 계산 적용으로 학습 안정성 확보



샘플별 정규화



진폭 표준화



입력 형태 변환



데이터 분리

5가지 CNN 모델 비교 실험

여러 CNN 구조를 비교하여 최적 모델을 선정하였으며, 필터 수, 레이어 깊이, 정규화 기법 등을 다양하게 조합하였습니다.

Model 1 (Baseline)

Conv1D + MaxPooling + Dropout 기본 구조로 구성하여 기본 성능을 측정하는 베이스라인 모델입니다.

Model 2 (Filter up)

Baseline 대비 Conv1D 필터 수를 확장하여 다양한 파형 특징을 추출할 수 있도록 설계하였습니다.

Model 3 (Deep Layer)

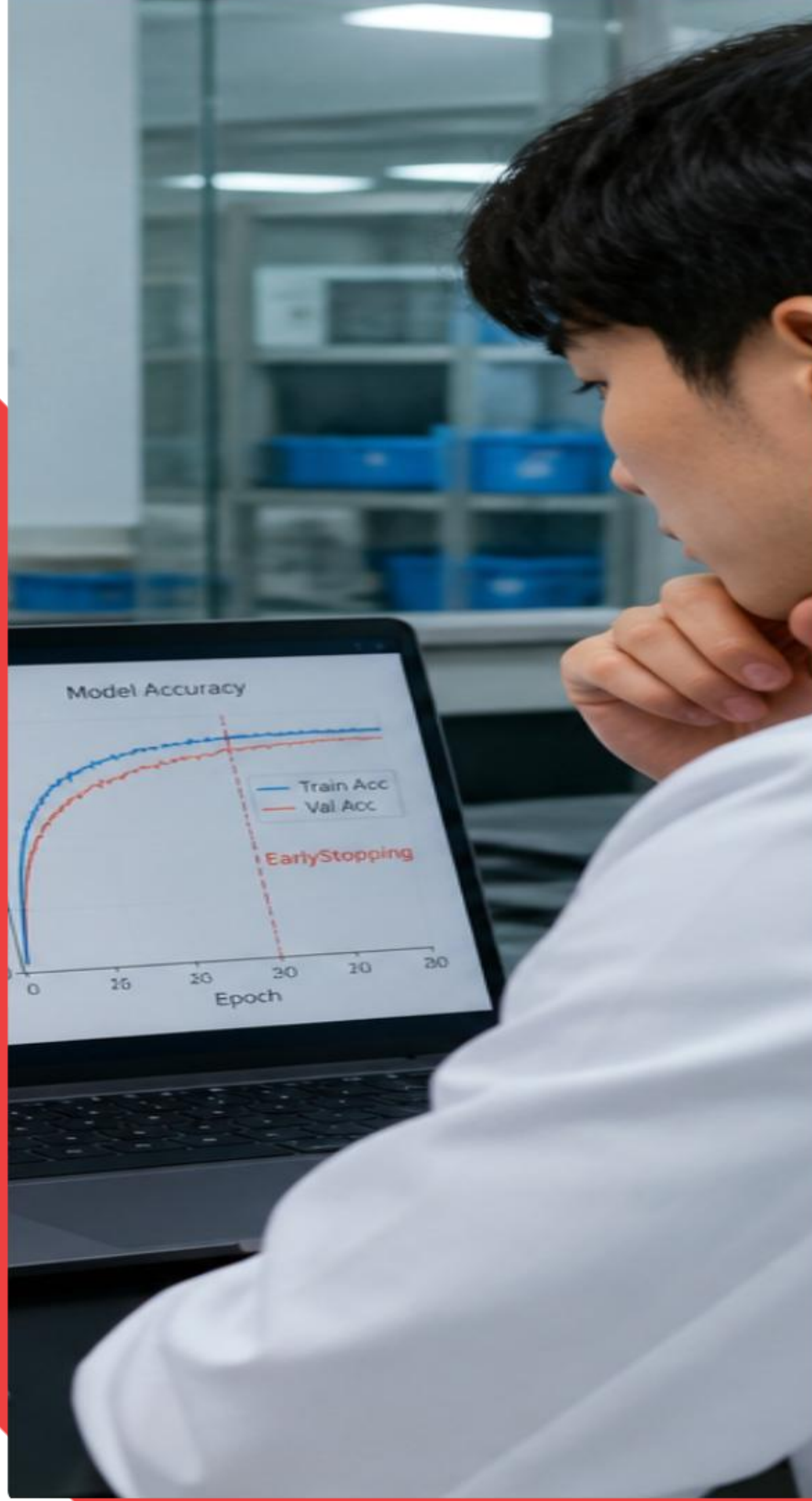
합성곱 레이어의 깊이를 추가하여 복잡한 패턴을 학습할 수 있는 심층 구조로 구현하였습니다.

Model 4 (BatchNorm)

BatchNormalization을 적용하여 학습 속도 및 안정성을 개선한 모델입니다.

Model 5 (Best Candidate)

필터 증가 + BatchNorm + Dropout을 조합하여 최적의 성능을 도출하는 후보 모델입니다.



모델 학습 전략



과적합 방지 및 학습 안정성 확보를 위한 전략

- EarlyStopping: 검증 손실 개선 없으면 조기 종료하여 과적합 방지
- ReduceLROnPlateau: 검증 손실 정체 시 LR 감소로 안정적 수렴 유도
- Class weight: 클래스 불균형 대응해 정상/부정맥 균형 학습
- 5개 모델을 동일 train/test로 학습·평가하여 공정 비교



Early Stopping



LR 조정



Class Weight



동일 데이터

각 모델 생성 코드(1)

```
# Model 1 : Baseline
if model_type == "baseline":

    model.add(Conv1D(
        16,
        kernel_size=21,
        activation="relu",
        padding="same",
        kernel_regularizer=l2(0.001),
        input_shape=(X.shape[1], 1)
    ))
    model.add(MaxPooling1D(8))
    model.add(Dropout(0.2))

    model.add(Conv1D(
        32,
        kernel_size=15,
        activation="relu",
        padding="same",
        kernel_regularizer=l2(0.001)
    ))
    model.add(MaxPooling1D(4))
    model.add(Dropout(0.25))

    model.add(Conv1D(
        64,
        kernel_size=9,
        activation="relu",
        padding="same",
        kernel_regularizer=l2(0.001)
    ))
    model.add(MaxPooling1D(2))
    model.add(Dropout(0.3))

    dense_units = 32
    dense_dropout = 0.3
    lr = 0.0003
```

Model 1(Baseline)

```
# Model 2 : Filter Up
elif model_type == "filter_up":

    model.add(Conv1D(
        32,
        kernel_size=21,
        activation="relu",
        padding="same",
        kernel_regularizer=l2(0.001),
        input_shape=(X.shape[1], 1)
    ))
    model.add(MaxPooling1D(8))
    model.add(Dropout(0.2))

    model.add(Conv1D(
        64,
        kernel_size=15,
        activation="relu",
        padding="same",
        kernel_regularizer=l2(0.001)
    ))
    model.add(MaxPooling1D(4))
    model.add(Dropout(0.25))

    model.add(Conv1D(
        128,
        kernel_size=9,
        activation="relu",
        padding="same",
        kernel_regularizer=l2(0.001)
    ))
    model.add(MaxPooling1D(2))
    model.add(Dropout(0.3))

    dense_units = 32
    dense_dropout = 0.3
    lr = 0.0003
```

Model 2(Filter Up)

```
# Model 3 : Deep Layer
elif model_type == "deep_layer":

    model.add(Conv1D(
        16,
        kernel_size=21,
        activation="relu",
        padding="same",
        kernel_regularizer=l2(0.001),
        input_shape=(X.shape[1], 1)
    ))
    model.add(MaxPooling1D(8))
    model.add(Dropout(0.2))

    model.add(Conv1D(
        32,
        kernel_size=15,
        activation="relu",
        padding="same",
        kernel_regularizer=l2(0.001)
    ))
    model.add(MaxPooling1D(4))
    model.add(Dropout(0.25))

    model.add(Conv1D(
        64,
        kernel_size=9,
        activation="relu",
        padding="same",
        kernel_regularizer=l2(0.001)
    ))
    model.add(MaxPooling1D(2))
    model.add(Dropout(0.3))

    model.add(Conv1D(
        128,
        kernel_size=5,
        activation="relu",
        padding="same",
        kernel_regularizer=l2(0.001)
    ))
    model.add(MaxPooling1D(2))
    model.add(Dropout(0.35))
    dense_units = 32
    dense_dropout = 0.3
    lr = 0.0003
```

Model 3(Deep Layer)

각 모델 생성 코드(2)

```
# Model 4 : BatchNorm
elif model_type == "batchnorm":

    model.add(Conv1D(
        32,
        kernel_size=21,
        activation="relu",
        padding="same",
        kernel_regularizer=l2(0.001),
        input_shape=(X.shape[1], 1)
    ))
    model.add(BatchNormalization())
    model.add(MaxPooling1D(8))
    model.add(Dropout(0.2))

    model.add(Conv1D(
        64,
        kernel_size=15,
        activation="relu",
        padding="same",
        kernel_regularizer=l2(0.001)
    ))
    model.add(BatchNormalization())
    model.add(MaxPooling1D(4))
    model.add(Dropout(0.25))

    model.add(Conv1D(
        128,
        kernel_size=9,
        activation="relu",
        padding="same",
        kernel_regularizer=l2(0.001)
    ))
    model.add(BatchNormalization())
    model.add(MaxPooling1D(2))
    model.add(Dropout(0.3))

    dense_units = 32
    dense_dropout = 0.3
    lr = 0.0003
```

Model 4(BatchNorm)

```
elif model_type == "best_candidate": # Model 5 : Best Candidate
    model.add(Conv1D(
        32,
        kernel_size=31,
        activation="relu",
        padding="same",
        kernel_regularizer=l2(0.001),
        input_shape=(X.shape[1], 1)
    ))
    model.add(BatchNormalization())
    model.add(MaxPooling1D(8))
    model.add(Dropout(0.25))
    model.add(Conv1D(
        64,
        kernel_size=21,
        activation="relu",
        padding="same",
        kernel_regularizer=l2(0.001)
    ))
    model.add(BatchNormalization())
    model.add(MaxPooling1D(4))
    model.add(Dropout(0.3))
    model.add(Conv1D(
        128,
        kernel_size=11,
        activation="relu",
        padding="same",
        kernel_regularizer=l2(0.001)
    ))
    model.add(BatchNormalization())
    model.add(MaxPooling1D(2))
    model.add(Dropout(0.35))

    model.add(Conv1D(
        256,
        kernel_size=5,
        activation="relu",
        padding="same",
        kernel_regularizer=l2(0.001)
    ))
    model.add(BatchNormalization())
    model.add(MaxPooling1D(2))
    model.add(Dropout(0.4))
    dense_units = 32
    dense_dropout = 0.3
    lr = 0.0003
```

Model 5(Best Candidate)

```
# 공통 구조
model.add(GlobalAveragePooling1D())

model.add(Dense(
    dense_units,
    activation="relu",
    kernel_regularizer=l2(0.001)
))

model.add(Dropout(dense_dropout))

model.add(Dense(1, activation="sigmoid"))

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=lr),
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

return model
```

공통 구조

모델 평가 지표

의료 AI 시스템에서는 단순 정확도를 넘어 부정맥 탐지 능력과 환자 안전을 종합적으로 평가하는 다양한 지표가 필요합니다.

Accuracy (정확도)

전체 예측 중 올바르게 분류한 비율을 나타내며, 모델의 전반적인 성능을 측정하는 기본 지표입니다.

Precision (정밀도)

부정맥이라고 예측한 것 중 실제로 부정맥인 비율을 의미하며, 오탐지(False Positive)를 줄이는 데 중요한 지표입니다.

Recall (재현율)

실제 부정맥 환자 중 모델이 정확히 탐지한 비율로, 의료 AI에서 가장 중요한 지표이며 오진(False Negative) 방지에 핵심적입니다.

F1-score

Precision과 Recall의 조화 평균으로, 두 지표 간의 균형을 평가하여 모델의 종합적인 분류 성능을 나타냅니다.

AUC

정상과 부정맥을 구분하는 모델의 종합적인 능력을 나타내며, ROC 곡선 아래 면적으로 측정됩니다.

최적 모델 선정 기준

의료 AI 특성을 반영하여 단순 정확도가 아닌 부정맥 탐지 능력과 환자 안전을 최우선으로 고려한 가중 평가 방식을 적용하였습니다.



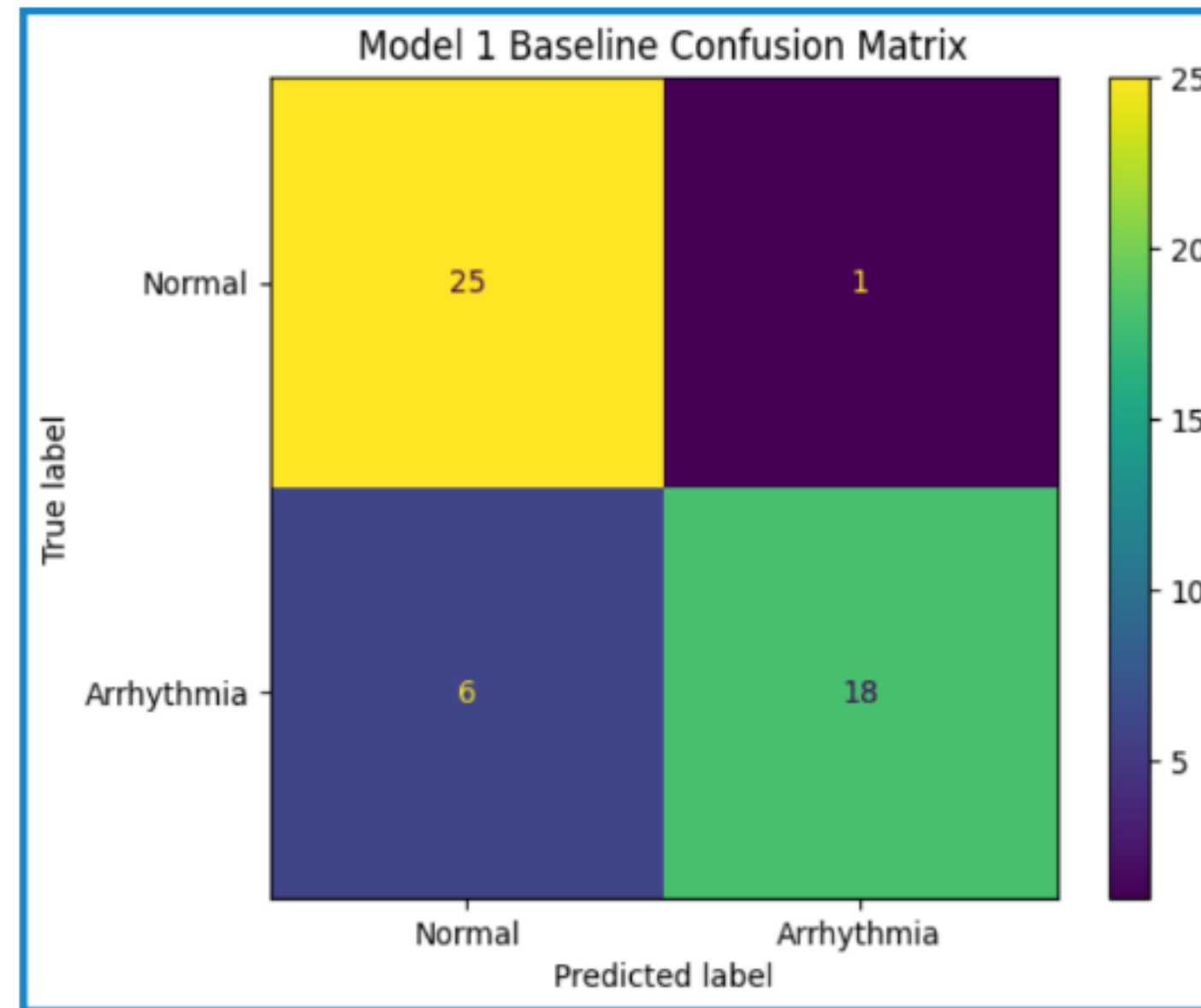
- AUC 40% 비중
- Recall 20% 비중
- 종합 구분 능력 강조
- 실제 부정맥 탐지 중시

- False Negative 최소화
- 부정맥 환자 누락 방지
 - 의료 안전성 확보
 - 진단 신뢰도 향상

- 환자 안전 최우선
- 균형 잡힌 평가
- 실용적 모델 선정
- 임상 적용 가능성

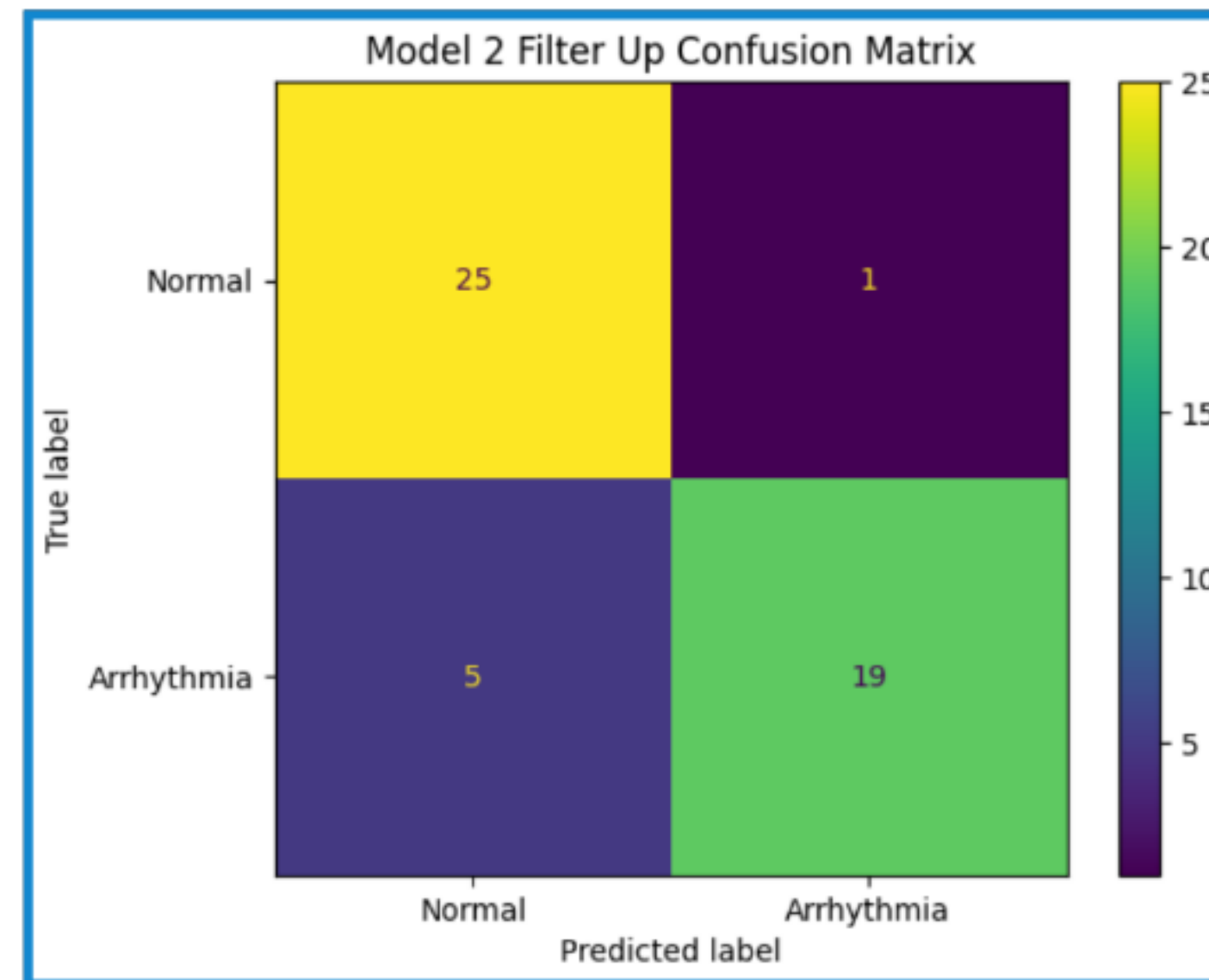
$$\text{Final Score} = \text{AUC} \times 0.4 + \text{F1-score} \times 0.3 + \text{Recall} \times 0.2 + \text{Accuracy} \times 0.1$$

각 모델의 평가표(1)



Model 1(Baseline)

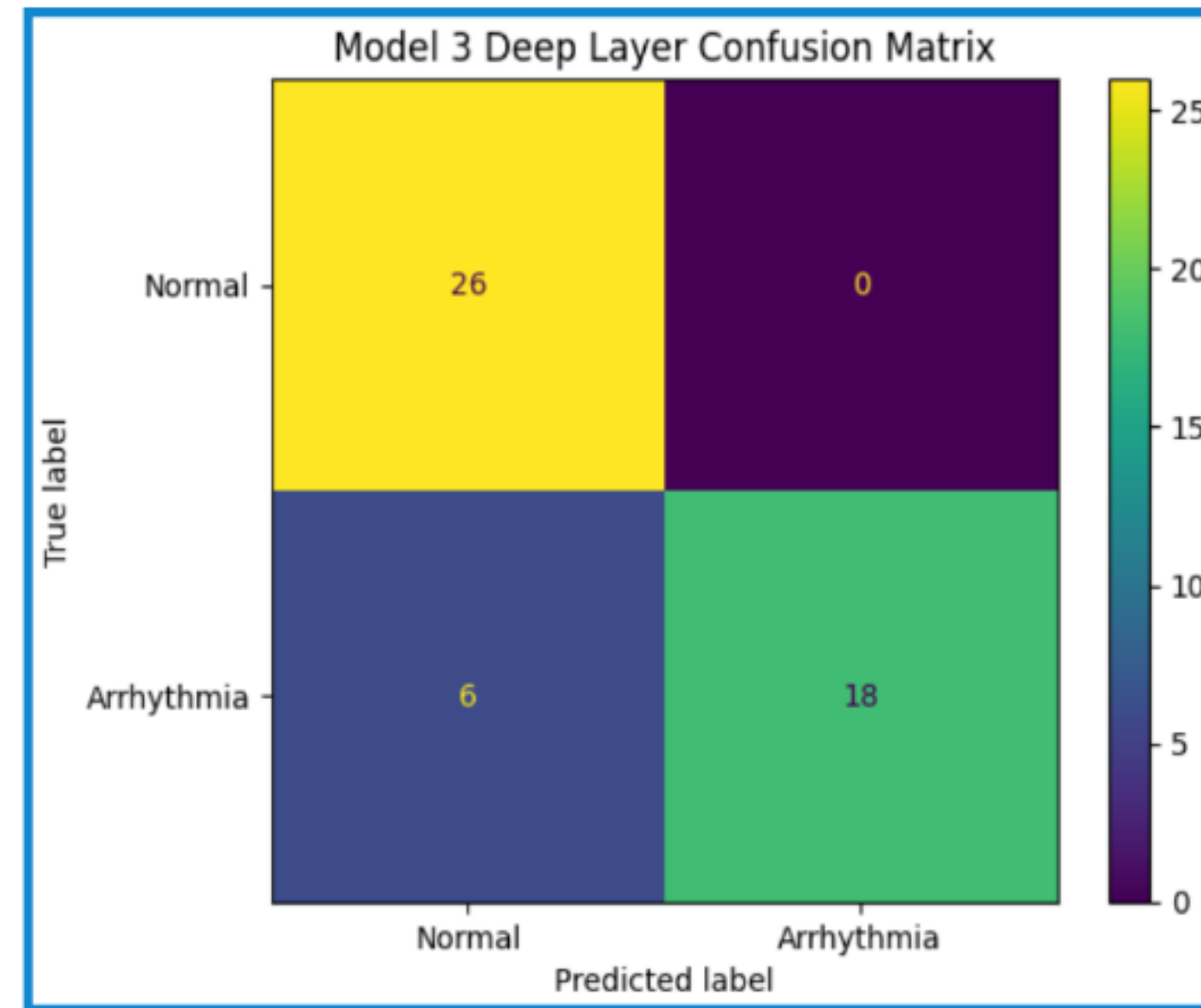
```
Model 1 Baseline
Accuracy      : 0.8600
Precision     : 0.9474
Sensitivity   : 0.7500
F1-score      : 0.8372
AUC           : 0.9423
Final Score   : 0.8641
```



Model 2(Filter Up)

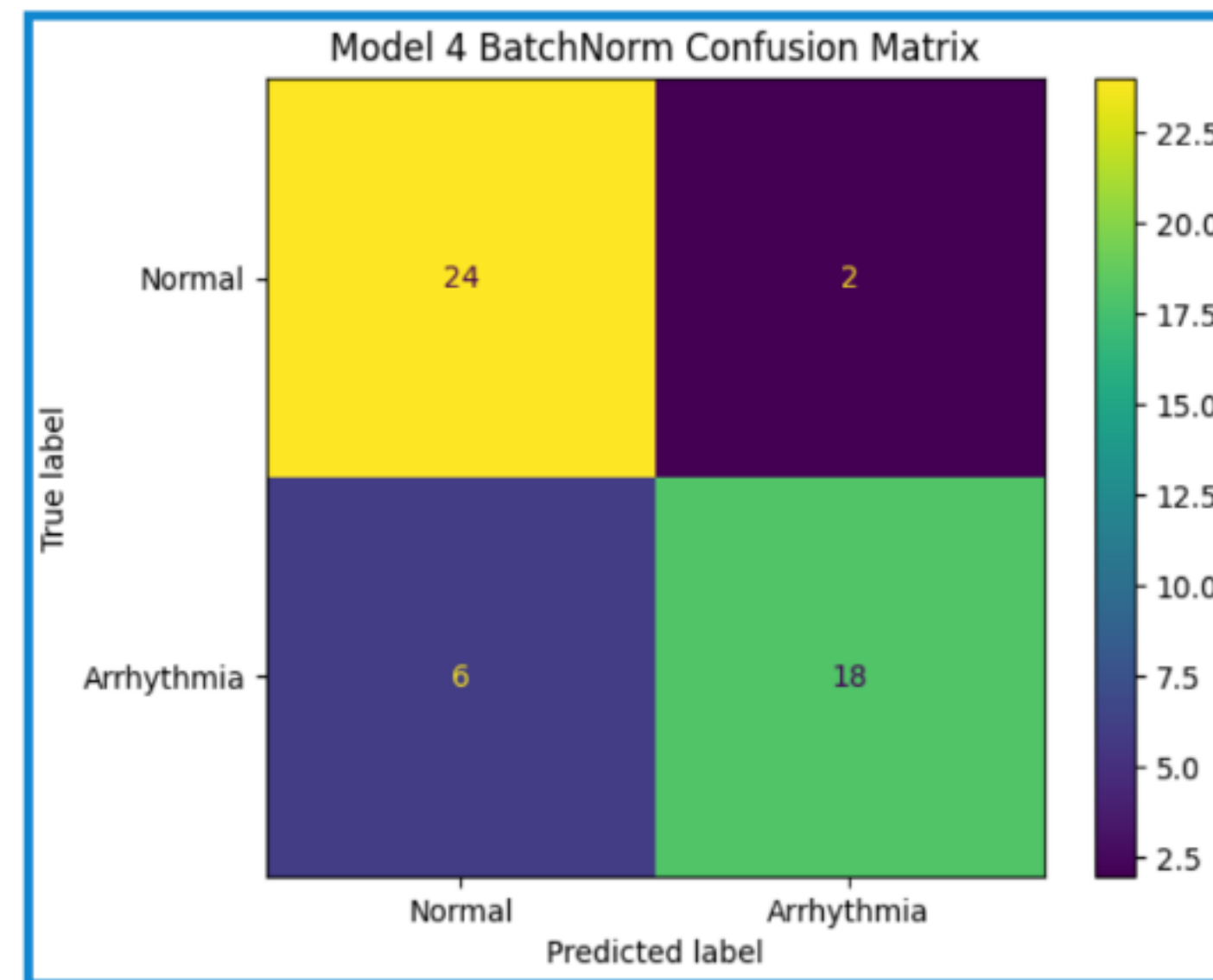
```
Model 2 Filter Up
Accuracy      : 0.8800
Precision     : 0.9500
Sensitivity   : 0.7917
F1-score      : 0.8636
AUC           : 0.9247
Final Score   : 0.8753
```

각 모델의 평가표(2)



Model 3(Deep Layer)

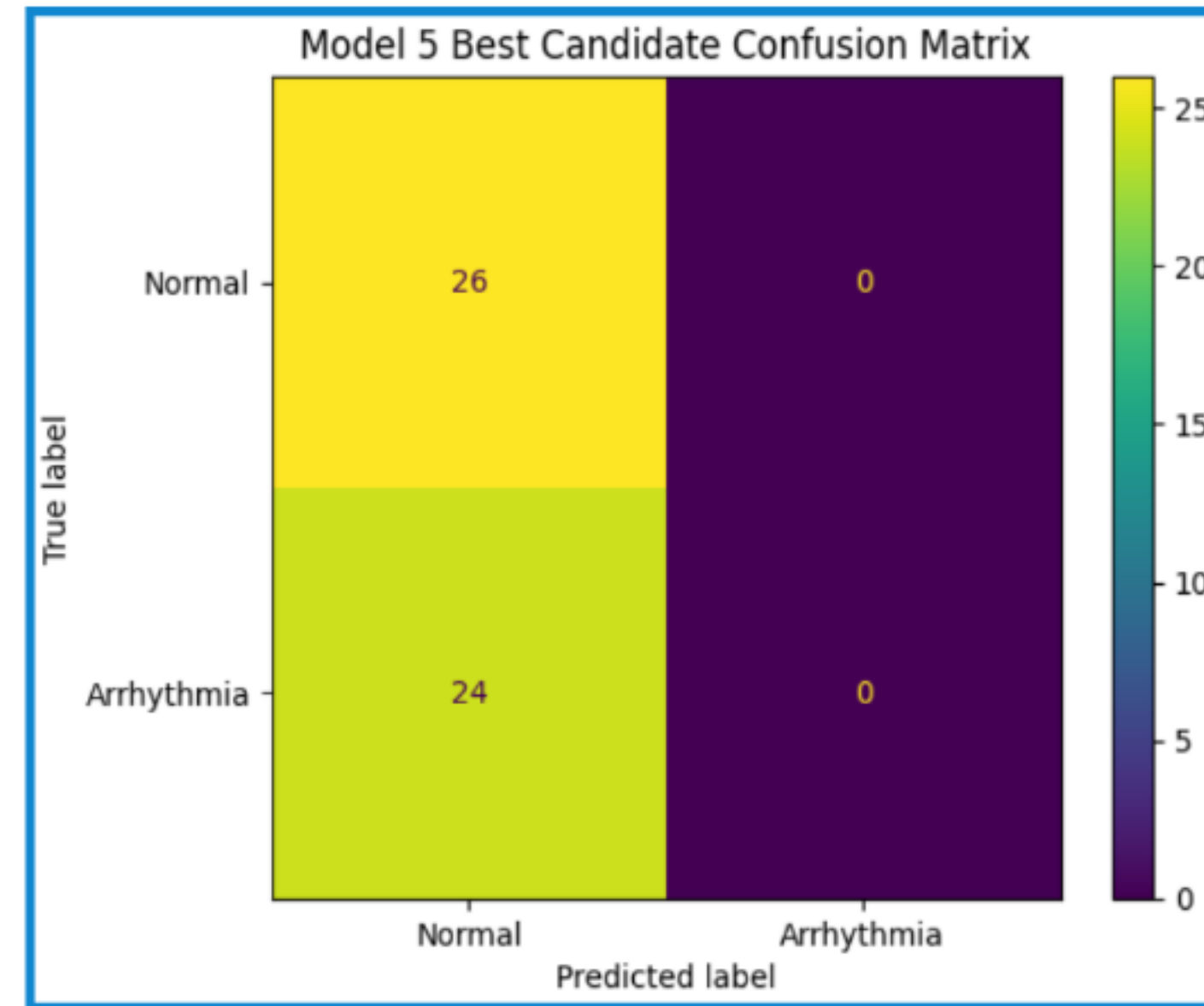
```
Model 3 Deep Layer
Accuracy      : 0.8800
Precision     : 1.0000
Sensitivity   : 0.7500
F1-score      : 0.8571
AUC           : 0.9375
Final Score   : 0.8701
```



Model 4(BatchNorm)

```
Model 4 BatchNorm
Accuracy      : 0.8400
Precision     : 0.9000
Sensitivity   : 0.7500
F1-score      : 0.8182
AUC           : 0.9295
Final Score   : 0.8512
```

각 모델의 평가표(3)

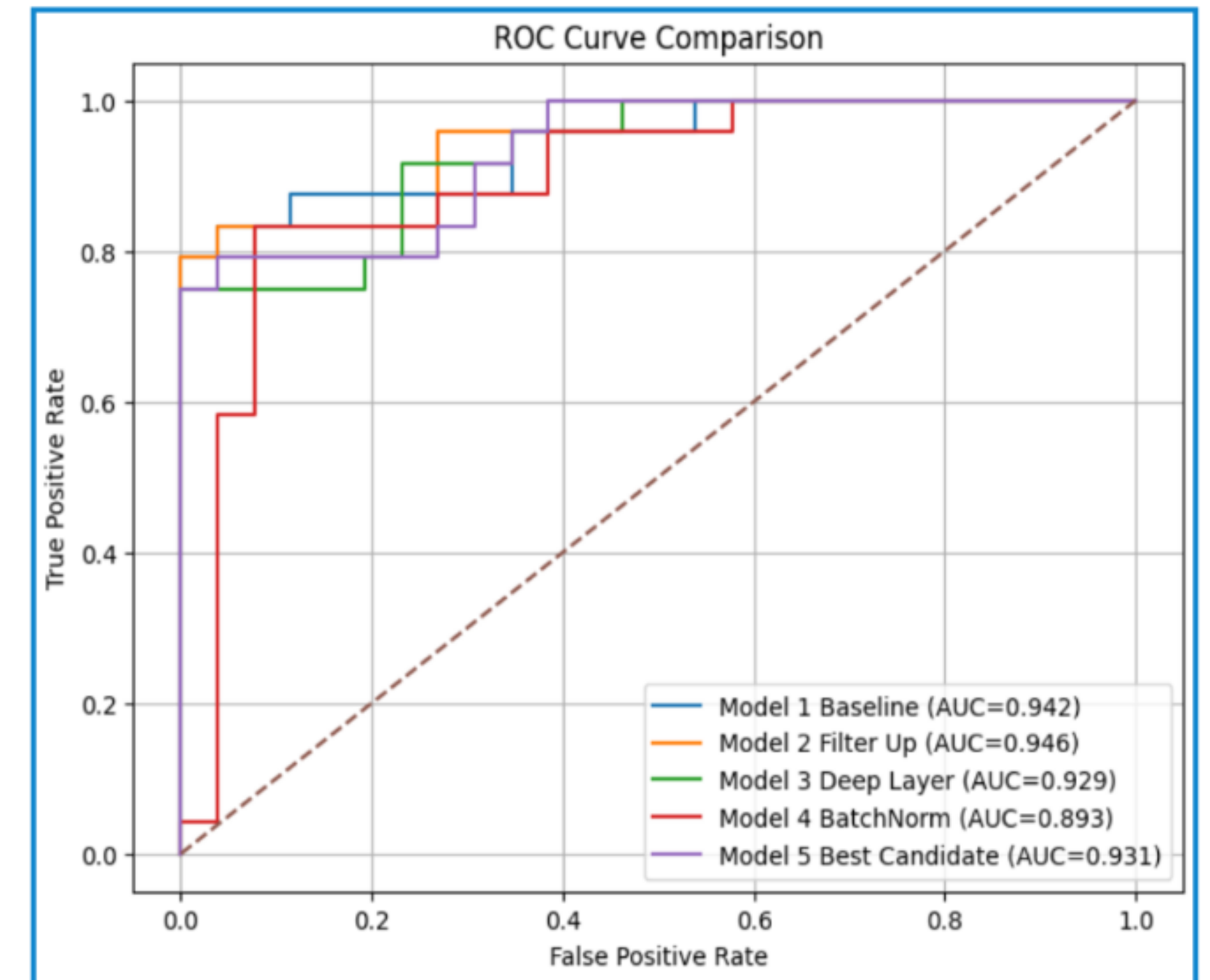
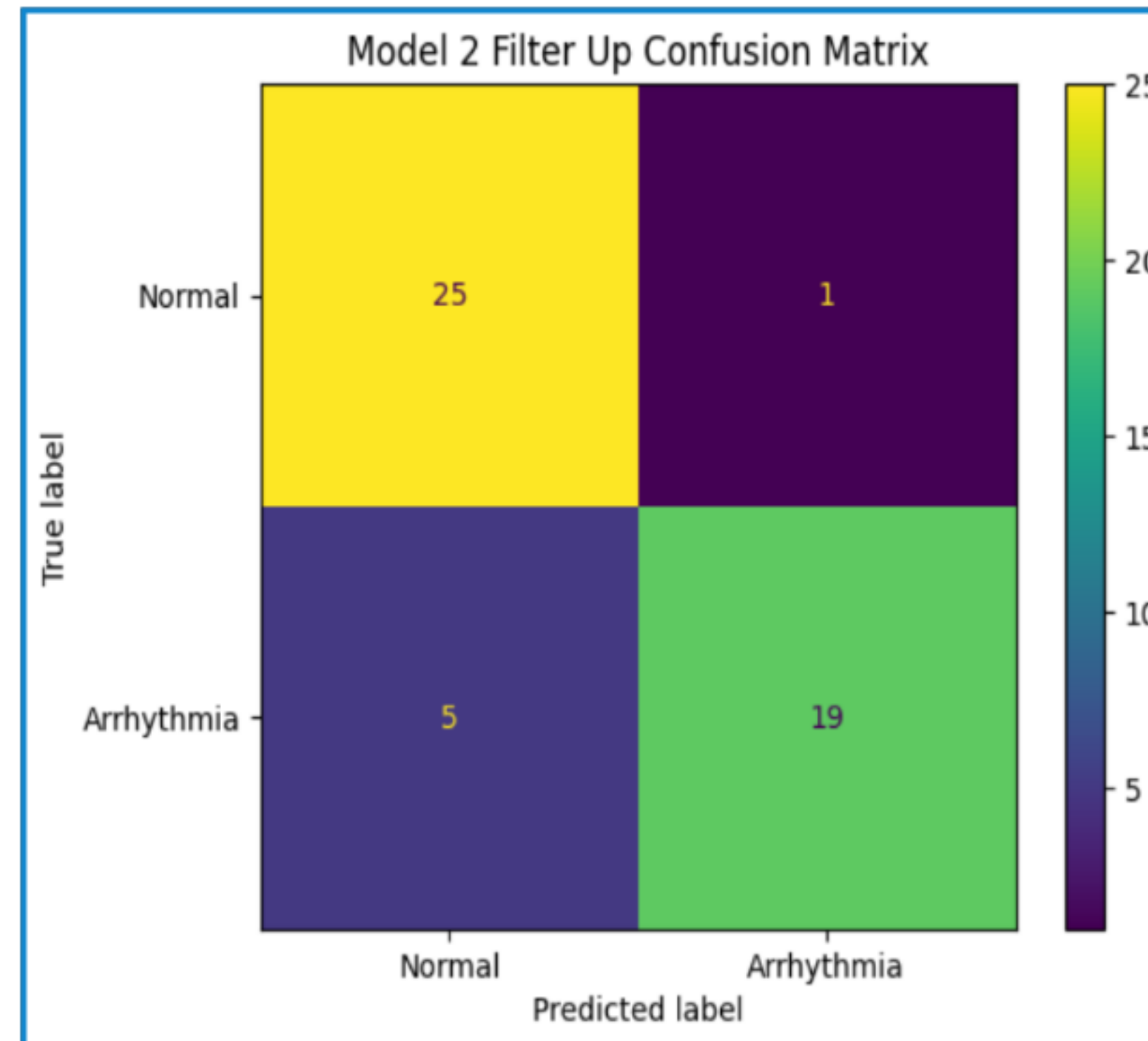


```
Model 5 Best Candidate
Accuracy      : 0.5200
Precision     : 0.0000
Sensitivity   : 0.0000
F1-score      : 0.0000
AUC           : 0.9199
Final Score   : 0.4199
```

Model 5(Best Candidate)

```
=====
Best Model : Model 2 Filter Up
Accuracy   : 0.8800
Precision  : 0.9500
Sensitivity : 0.7917
F1-score   : 0.8636
AUC        : 0.9247
Final Score : 0.8753
=====
최고 성능 모델 저장 완료
저장된 모델 : Model 2 Filter Up
```

각 모델의 ROC Curve 비교 & ConfusionM atrix



좌: Best Model(Filter up)의 ConfusionMatrix 우: Model 1 ~ Model 5의 Roc Curve 비교

모델 평가 결과

다양한 CNN 구조를 체계적으로 비교
평가하여 최고 성능의 모델을 선정하
고, 실제 웹 시스템에 통합하였습니다.

Confusion Matrix 및 ROC Curve

- 5개 모델의 혼동 행렬 분석
- ROC 곡선 비교를 통한 성능 시각화
- 각 모델의 강점과 약점 파악

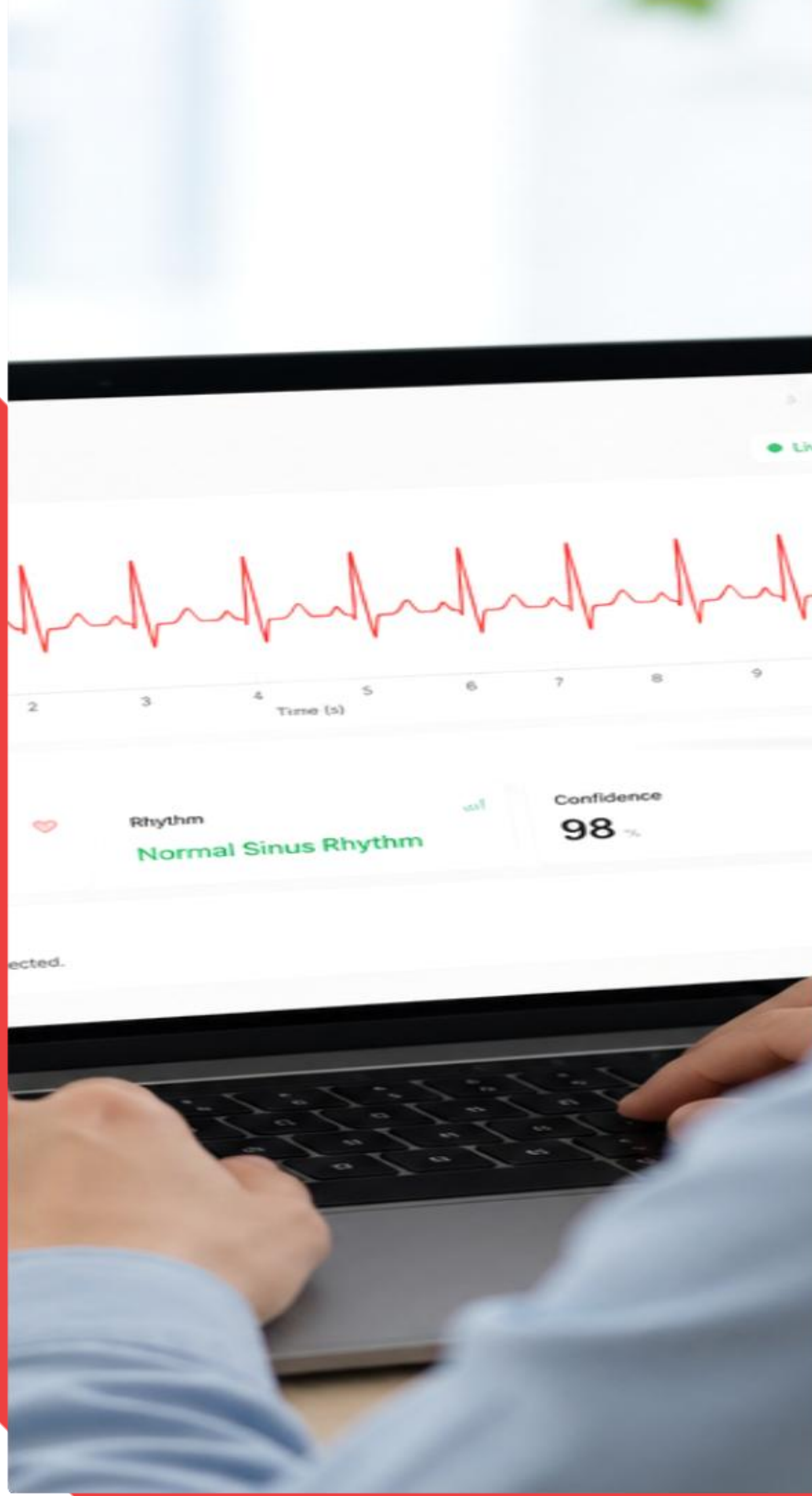
5가지 CNN 모델의 종합 평가 및 최적 모델 도출 = Filter up 모델

Final Score 기반 선정

- Final Score 최고점 모델 선정
- AUC, Recall, F1-score 종합 고려
- 의료 AI 특성 반영한 최적화

모델 저장 및 배포

- ecg_cnn_10sec_tuned.keras 저장
- Streamlit 연동
- load_model()로 실시간 분석



Streamlit 기반 웹 시스템 구현



학습된 CNN 모델의 실용화 및 사용자 접근성 확보

- 학습된 CNN 모델을 웹 애플리케이션으로 구현
- 실시간 ECG 그래프 및 AI 분석 결과 즉시 제공
- 사용자 친화적 UI로 비전문가도 직관적 이해 가능
- 연구 기반을 실서비스 형태로 확장하여 헬스케어 플랫폼화



실시간 분석



직관적 UI



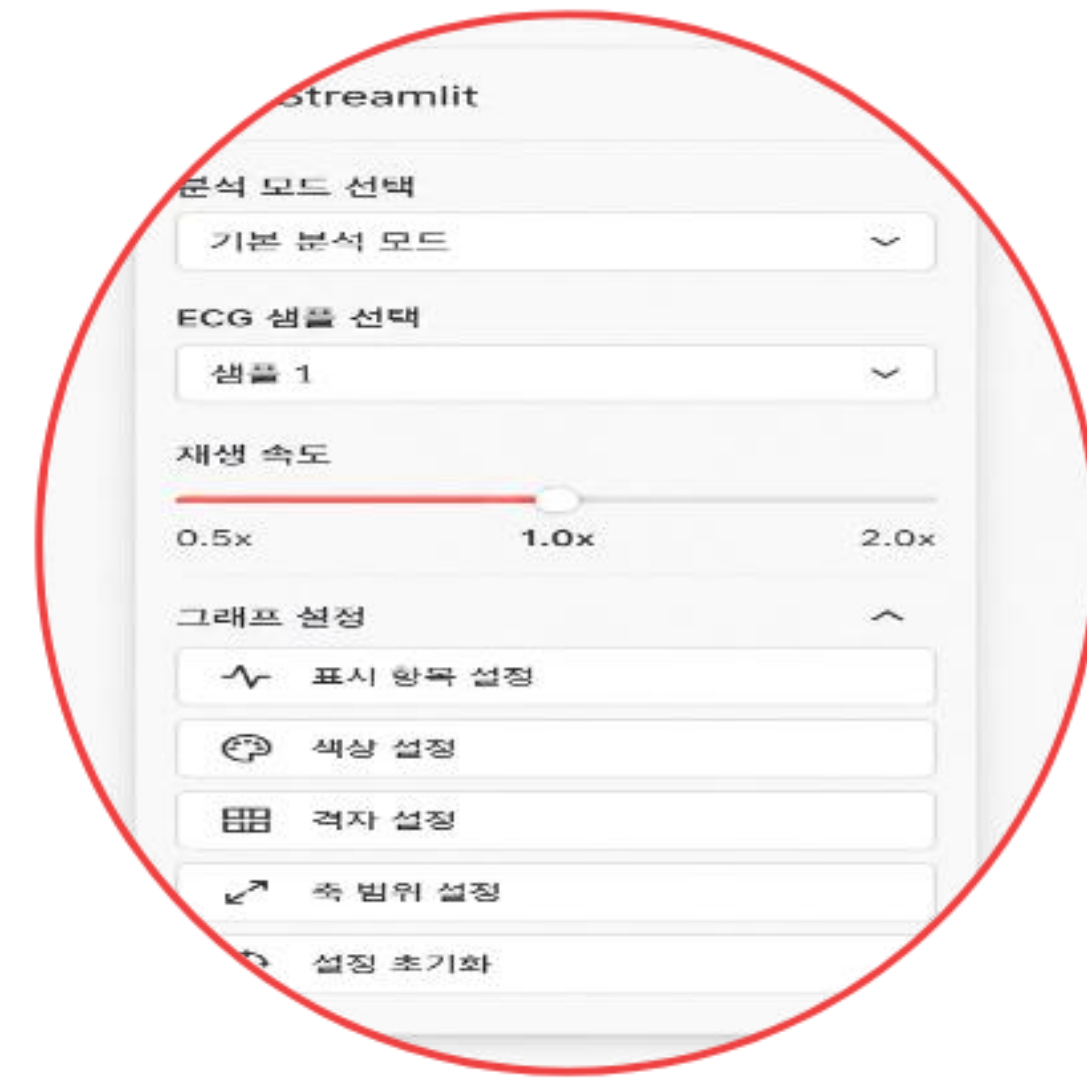
그래프 시각화



한계 극복

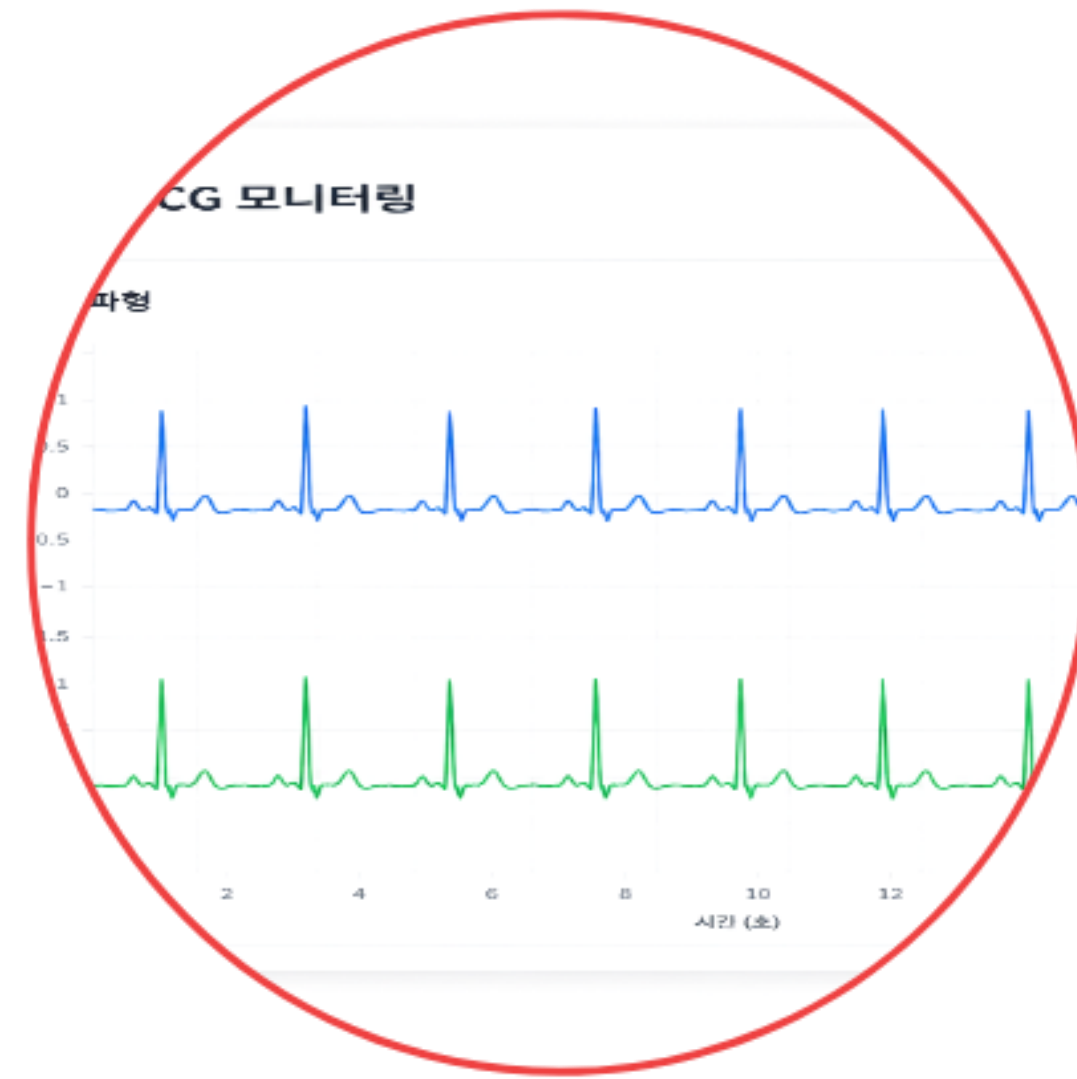
웹 시스템 구성

Streamlit 웹 시스템은 사용자 설정, 실시간 ECG 파형 시각화, AI 분석 결과 제공의 세 영역으로 구성되어 직관적인 사용 경험을 제공합니다.



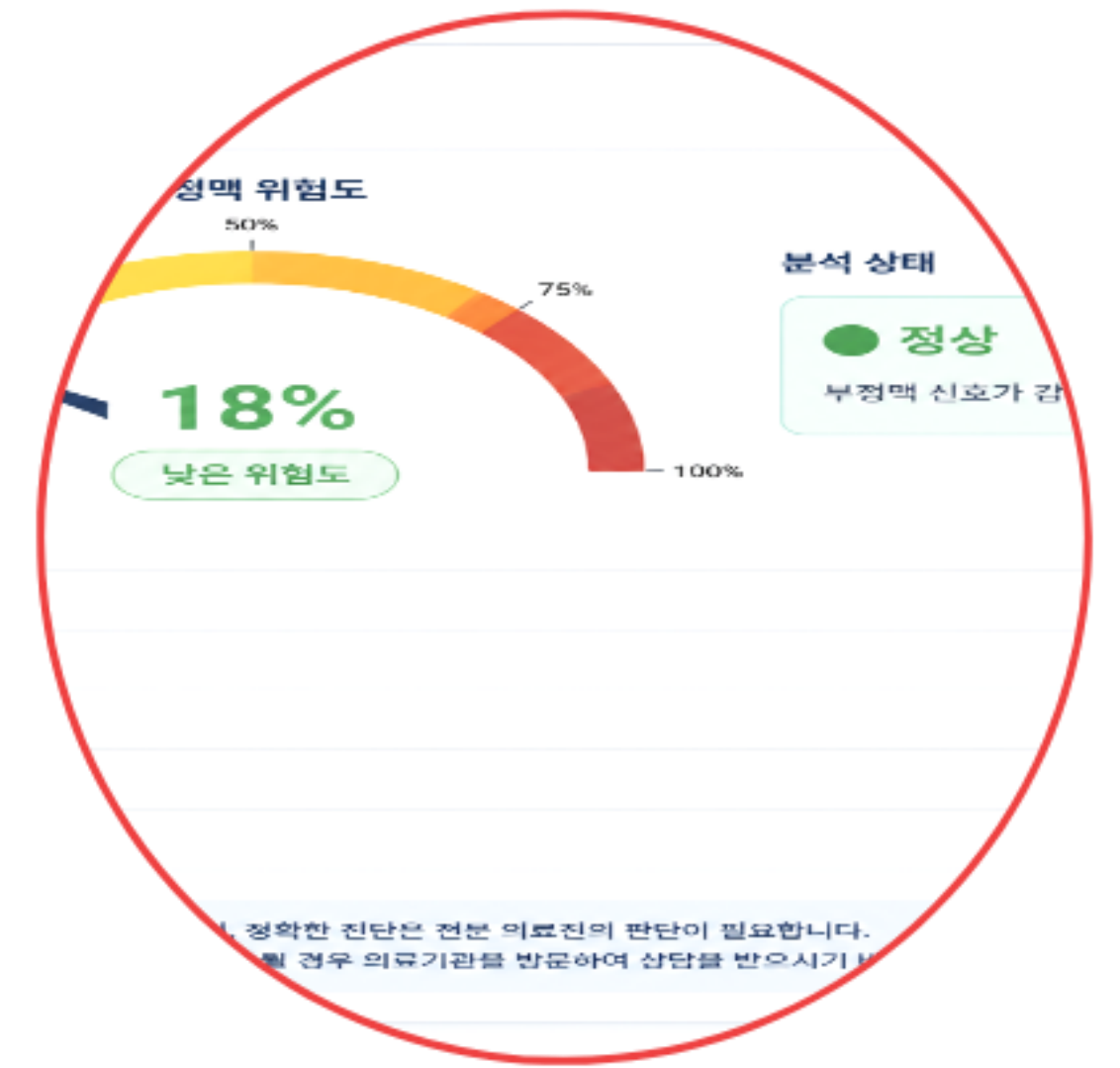
왼쪽 사이드바: 설정 영역

- 분석 모드 선택: 기존 샘플 분석 또는 파일 업로드 모드
- ECG 샘플 선택: 환자별 데이터 선택 기능
- 재생 속도 조절: 사용자 맞춤형 시각화 속도 설정
- 그래프 진행 단위 설정: 세밀한 파형 관찰 가능
- 직관적인 인터페이스로 손쉬운 조작



중앙 영역: ECG 시각화

- Plotly 라이브러리 활용한 동적 그래프 생성
- ECG 신호를 실시간처럼 점진적으로 출력
 - 시작 버튼으로 파형 재생 제어
- 시간에 따른 심전도 변화 시각적 확인
- 실제 ECG 모니터링 시스템과 유사한 경험 제공



오른쪽 영역: 분석 결과

- CNN 모델의 부정맥 분석 결과 표시
 - 부정맥 위험도 0~100% 수치화
 - 정상, 부정맥 의심 단계별 분류
 - 상세 정보 및 해석 제공
- 사용자가 자신의 ECG 상태를 직관적으로 이해

업로드 기능

기존 데이터셋 샘플 선택 방식의 한계를 극복하고, 사용자가 직접 보유한 ECG 데이터를 업로드하여 분석할 수 있는 확장 기능을 구현하였습니다.

CSV/NPY 파일 지원

- CSV/NPY 형식 ECG 파일 업로드 지원
- 다양한 데이터 소스 호환
- 외부 ECG 데이터 분석 가능

ECG 파일 업로드 분석 기능

자동 분석 처리

- 업로드된 파일 자동 파싱 및 전처리
- CNN 모델 적용하여 즉시 분석
- 데이터 형식 검증 및 오류 처리

실시간 결과 제공

- 부정맥 위험도 실시간 계산
- 파형 시각화 및 분석 결과 동시 제공
- 직관적인 UI로 결과 확인



과제 결과물의 특징점 (1)



CNN 모델과 웹 기반 실시간 시각화 시스템의 통합

- Plotly 기반 동적 그래프로 실시간 ECG 파형 시각화
- 학습된 CNN으로 부정맥 가능성을 0~100% 위험도로 변환
- 결과와 위험도를 함께 표시해 비전문가도 직관적 확인
- 실제 모니터링 유사 UX로 의료 현장 적용 가능
- 실사용 가능한 웹 서비스 형태로 구현



실시간 시각화



위험도 수치화



직관적 UI



사용자 경험

특장점 (2)

본 과제는 단순 알고리즘 연구를 넘어 실제 사용 가능한 통합 시스템을 구현하여 즉각 배포 및 사업화가 가능한 실무적 가치를 확보하였습니다.



확장성 및 유연성

- ECG 파일 업로드로 신규 데이터 분석 가능
- CSV/NPY 지원으로 다양한 소스 호환
- 웨어러블·의료장비 연동 가능성 확보
 - 실제 사용 환경에서 활용성 확대
 - 사용자 맞춤형 분석 서비스 제공
- 지속적 데이터 축적으로 모델 개선 기반 마련
- 확장 가능한 아키텍처로 기능 추가 용이



최적 모델 선정 프로세스

- 5개 CNN 구조 비교로 최적 모델 선정
- Accuracy, Precision, Recall, F1, AUC 등 지표 종합 고려
- 의료 AI 특성 반영한 Final Score 적용
- 부정맥 탐지 능력 중심 평가로 환자 안전 우선
- EarlyStopping·ReduceLROnPlateau로 과적합 방지
- 검증된 최적 모델을 웹 시스템에 통합 배포



실제 시스템 유사성

- 실제 모니터링 유사 UI 구현
- 시작 버튼으로 ECG 파형 점진 출력되는 실시간 경험 제공
- 의료장비 시각화 방식 반영한 직관적 UI 설계
- 분석 결과와 파형을 동시에 확인 가능한 통합 화면
- 재생 속도·그래프 진행 단위 조절로 사용자 편의성 향상
- 실용적 헬스케어 플랫폼으로 완성도 확보

과제 요약

본 과제는 MIT-BIH 기반 ECG 부정맥 분석 시스템을 개발하여 데이터 구축에서 AI 모델링, 웹 서비스 배포까지 전 과정을 통합 구현하였습니다.

데이터 구축

- MIT-BIH Arrhythmia Database에서 환자 5명(200, 208, 210, 214, 221번)의 ECG 데이터 추출
- 10초 단위 window 분할 방식 적용하여 총 250개 샘플 생성 (환자당 50개)

모델 개발

- PVC 개수 기준 라벨링: 0~2개는 정상(128개), 3개 이상은 부정맥(122개)
- 클래스 균형 유지로 모델 편향 문제 최소화 (정상 51.2%, 부정맥 48.8%)

웹 시스템 구현

- Conv1D 기반 5가지 CNN 구조 설계 및 비교 실험 수행
- Accuracy, Recall, F1-score, AUC 등 다양한 평가 지표 종합 고려하여 최적 모델 선정

분석 기능 확장

- Streamlit 기반 웹 UI 구현으로 실시간 ECG 신호 그래프 출력
- Plotly 활용 동적 시각화 및 부정맥 위험도 0~100% 수치화 제공

통합 파이프라인

- CSV/NPY 파일 업로드 분석 기능 구현으로 외부 ECG 데이터 분석 가능
- 사용자 중심 인터페이스로 비전문가도 직관적으로 심혈관 건강 모니터링 가능

활용 방안(1)

본 시스템은 스마트워치·패치형 웨어러블과 연동되어 일상 심혈관 상태를 실시간 모니터링하는 예방 중심 헬스케어 플랫폼으로 확장됩니다.

웨어러블 기기 연동

- 스마트워치·패치형 ECG 등 연동
- 생체신호 장비 데이터 실시간 수집
- IoT 기반 통합 헬스케어 구축

디지털 헬스케어 플랫폼 확장

실시간 모니터링

- 24시간 심혈관 상태 상시 모니터링
- 부정맥 발생 즉시 알림 및 경고
- 이상 징후 조기 발견 및 대응

예방 중심 서비스

- 질병 발생 전 예방적 건강 관리
- 개인 맞춤형 건강 관리 서비스 제공
- 만성 질환자 지속 관리 플랫폼



활용 방안 (2)



의료진 판독 부담 경감 및 진단 효율화

- 대용량 ECG 신호에서 부정맥 의심 구간을 AI가 1차적으로 자동 탐지 및 선별
- 1차 의료 기관에서 사전 스크리닝 도구로 활용하여 고위험 환자 우선 식별
- 전문 의료진의 판독 시간 및 비용 부담을 획기적으로 경감
- 진단 효율화로 더 많은 환자에게 신속한 의료 서비스 제공 가능
- 의료 서비스 품질 향상 및 환자 안전성 증대



자동 탐지



스크리닝



효율화



품질 향상

통합 서비스의 실무 가치

단순 알고리즘 연구를 넘어 실제 사용
자가 활용 가능한 통합 플랫폼을 구축
하였습니다.



End-to-End 웹 서비스 구현

- 외부 파일 업로드 기능 지원
- 실시간 ECG 파형 시각화
- 부정맥 위험도 자동 도출
- 직관적 웹 인터페이스 제공
- CSV/NPY 파일 형식 지원
- Streamlit 기반 경량 배포
- 사용자 친화적 UI/UX 설계
- 전 과정 자동화 파이프라인



즉각 배포 가능한 시스템

- 학습된 CNN 모델 즉시 배포
 - 별도 설치 없는 웹 접근
- 클라우드 기반 확장 가능
- 실시간 분석 결과 제공
 - 높은 상용화 준비도
- 의료 현장 적용 가능
 - 비용 효율적 구조
- 유지보수 용이성 확보



기술-서비스 융합 가치

- AI 모델과 웹 시스템 통합
- 연구 성과의 실용화 증명
 - 헬스케어 서비스 혁신
- 디지털 전환 사례 제시
 - 사업화 가능성 입증
- 기술 이전 준비 완료
- 실무 적용 가치 극대화
 - 산학협력 성과 창출

기대효과

본 시스템은 의료 현장과 일상 건강 관리 전반에 걸쳐 실질적 가치를 제공합니다.

심혈관 질환 조기 발견 및 예방

- 부정맥 자동 탐지로 심정지·뇌졸중 등 치명적 합병증 사전 차단
- 고위험군 환자 조기 식별 및 적시 치료 개입으로 생존율 향상

의료진 업무 효율화

- 대용량 ECG 데이터 1차 스크리닝 자동화로 판독 시간 대폭 단축
- 의료진의 진단 부담 경감 및 의료 비용 절감, 진단 일관성 확보

건강 관리 접근성 향상

- 비전문가도 직관적으로 심혈관 상태 확인 가능한 사용자 중심 플랫폼
- 웨어러블 기기 연동 시 일상 속 실시간 모니터링 및 예방 관리 실현

AI 헬스케어 실용화

- 연구 단계 AI 모델을 실제 서비스로 전환한 성공 사례 제시
- 의료 AI 기술의 현장 적용 가능성 및 상용화 경로 구체화

디지털 헬스케어 산업 발전

- End-to-End 통합 시스템 구축으로 디지털 헬스케어 혁신 선도
- 고령화 사회 대응 및 예방 중심 의료 패러다임 전환에 기여

결론 및 향후 계획



AI 기반 ECG 부정맥 판별 시스템 성공적 개발

- MIT-BIH 데이터셋 기반 Conv1D CNN 모델 구현 및 최적화 완료
- Streamlit 웹 시스템 연동으로 실시간 ECG 분석 플랫폼 구축 성공
- 향후 더 많은 환자 데이터 확보 및 다양한 부정맥 유형 분류 기능 확장 예정
- 웨어러블 기기 연동 및 실제 의료 현장 적용을 통한 상용화 추진
- 지속적인 모델 성능 개선 및 사용자 피드백 반영으로 실용성 강화



데이터 확장



모델 고도화



기능 확장



현장 적용

Q&A



본 연구에 대한 질문과 의견을 자유롭게 나눠주시기 바랍니다.

- 시스템 구현 및 기술적 세부사항
 - 모델 성능 및 평가 방법
- 실제 적용 가능성 및 확장 방안

감사합니다.